# DAX Guidelines

*Personal conventions for writing DAX code*

## When it applies

Whenever DAX code is written, fixed, optimized, reviewed or explained: measures, calculated columns, calculated tables, CALCULATE, SUMX and SUMMARIZE/SUMMARIZECOLUMNS. It also applies when the context is a Power BI data model even if "DAX" isn't mentioned.

## Before writing

- Always check sqlbi.com first; no confirmation needed. Any other source (Microsoft docs, blogs, forums) needs approval first.

## Style and readability

- Always comment measures and any non-trivial logic block.
- Use `VAR` as much as possible, always prefixed with `_` (e.g. `VAR _TotalSales`).
- Columns created on the fly (e.g. in `ADDCOLUMNS ( SUMMARIZE ( ... ), "@ColumnName", ... )`) must be named with the `@` prefix and referenced as `[@ColumnName]`.
  - Why: tables created on the fly have no name, so the standard `TableName[ColumnName]` notation can't be used. A bare `[ColumnName]` would look like a measure reference; the `@` prefix removes that ambiguity.

## Variables are never recalculated

- `CALCULATE ( _Variable, Filter )` is dead code because the variable is already evaluated. To change the context, repeat the expression inside CALCULATE.
- A variable is fine as a filter argument, e.g. `CALCULATE ( SUM ( Sales[Amount] ), _QualifiedCustomers )`.

## CALCULATE and context transition

- Don't follow the "no CALCULATE" approach; use CALCULATE normally where needed.
- Remove filters with `REMOVEFILTERS`, never `ALL`.
- Never trigger a context transition on a fact table. On dimensions it's fine.

## Minimum columns

- Reference only the columns strictly needed, not entire tables.

## VALUES / SUMMARIZE / SUMMARIZECOLUMNS

- To group by one column, use `VALUES ( T[Col] )`, never `SUMMARIZE ( T, T[Col] )`.
- To group by two or more columns, use `SUMMARIZE ( T, T[Col1], T[Col2] )`.
  - It is for grouping only, with no aggregations inside.
  - Add aggregations with `ADDCOLUMNS ( SUMMARIZE ( ... ), "@...", ... )`.
- Use `SUMMARIZECOLUMNS` only when there's at least one aggregation.
  - No filters inside it.

  - Filter by wrapping it: `CALCULATETABLE ( SUMMARIZECOLUMNS ( ... ), Filters )`.

## Iterators

- At most one level of nesting. `SUMX ( T1, SUMX ( T2, ... ) )` is fine; a third level isn't. If you need more, restructure with variables or support tables.

## Naming

- A column and a measure never share a name, even on different tables.
- Measures are referenced without the table (`[Measure]`). Columns always include the table (`Table[Column]`).
- No measure, column or calculated table name starts with _, since that prefix is reserved for variables.
- No model object (measure, column, table, calculated column or calculated table) name starts with `@`, since that prefix is reserved for columns created on the fly. This keeps `[@ColumnName]` unambiguous.

## Final checklist

Before delivering code, every rule above gets checked:

- ☐ comments are present
- ☐ variables use _; columns created on the fly use `@` and are referenced as `[@ColumnName]`
- ☐ no VAR is used as an expression in CALCULATE or CALCULATETABLE
- ☐ REMOVEFILTERS is used instead of ALL
- ☐ no context transition on a fact table
- ☐ only the minimum columns are referenced
- ☐ VALUES vs SUMMARIZE is chosen by column count
- ☐ SUMMARIZECOLUMNS is used correctly
- ☐ iterators have at most one level of nesting
- ☐ names follow the naming rules: no model object starts with _ or `@`